BROWN
Computer Science

CS1010: Theory of Computation

Lecture 1: Deterministic Finite State Automata (DFA)

Lorenzo De Stefani

Fall 2020

Outline

- What is a Finite State Automaton
- DFA definition
- Example DFA construction
- The language of a DFA
- Regular Operations
- Closure under union
- Closure under concatenation

From Sipser Chapter 1.1

What is a Computer?

- We start with a simple computational model
  - Different models will have different features and may match a real computer better in some ways and worse in others
  - Trade off between generality and precision
- Our first model is the finite state machine or finite automaton
  - Model for machines with finite memory

Finite Automata

Models of computers with extremely limited memory

- Many simple computers have extremely limited memories and are, in fact, finite state machines
- Can you name any?
  - Vending machine
  - Elevator
  - Thermostat
  - Automatic door at supermarket

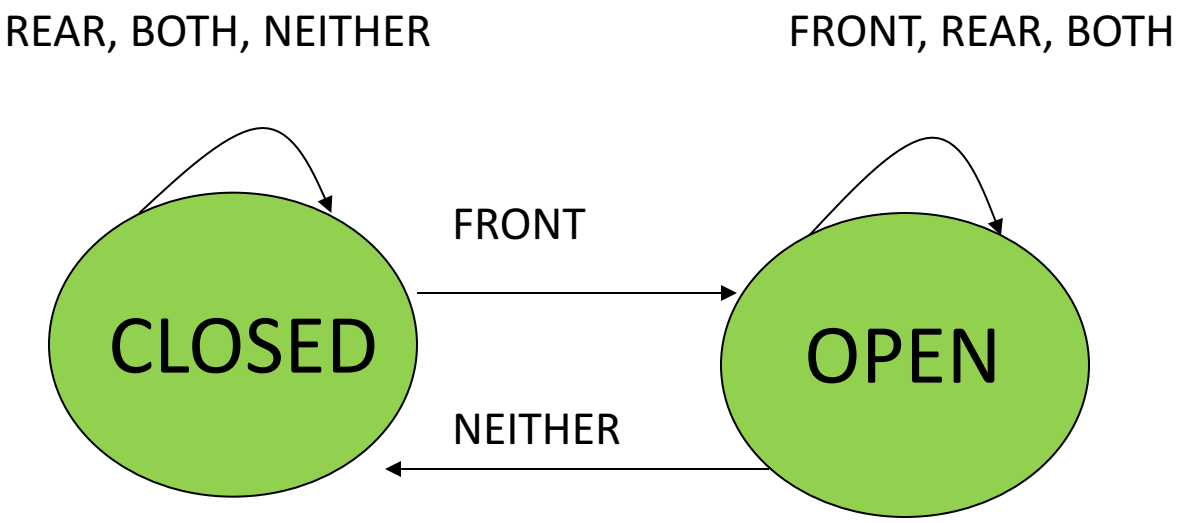
Automatic Door

- What is the desired behavior? Describe the actions and then list the states.
  - Person approaches, door should open
  - Door should stay open while person going through
  - Door should shut if no one near the doorway
  - States are open and closed

  } Input "signals/actions"

- More details about automatic door
  - Front pad Door Rear Pad
  - Describe behavior now:
    - Hint: action depends not just on what happens, but what state you are currently in
    - If you walk through, the door should stay open while you are on rear pad
    - But if door is closed and someone steps on rear pad, door does not open

Automatic Door

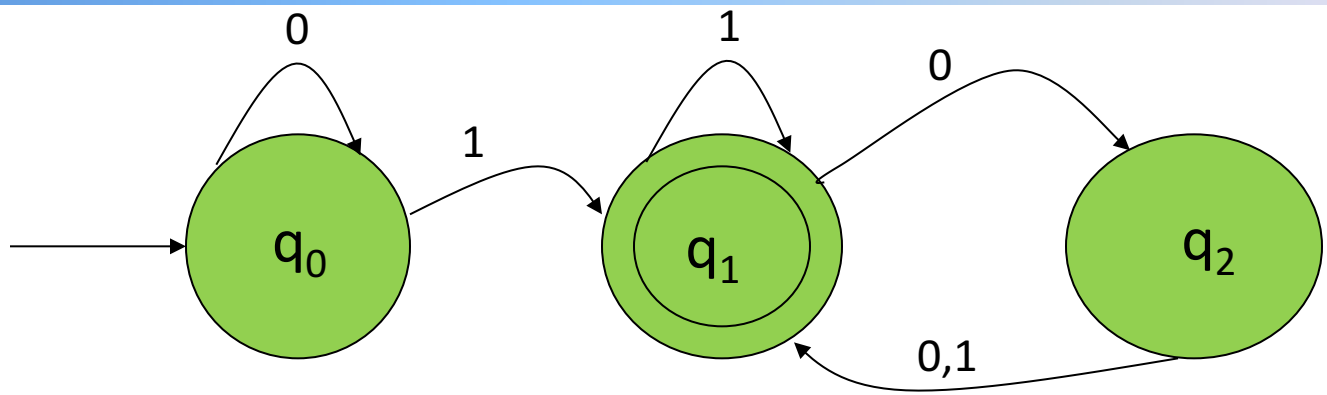


	NEITHER	FRONT	REAR	BOTH
CLOSED	CLOSED	OPEN	CLOSED	CLOSED
OPEN	CLOSED	OPEN	OPEN	OPEN

More on Finite Automata

- How many bits of data does this Finite States Machine (FSM) store?
  - 1 bit for states: open or closed
- What about state information for elevators, thermostats, vending machines, etc?
- FSM used in speech processing, optical character recognition, etc.

A finite automaton



A finite automaton M_1 with 3 states

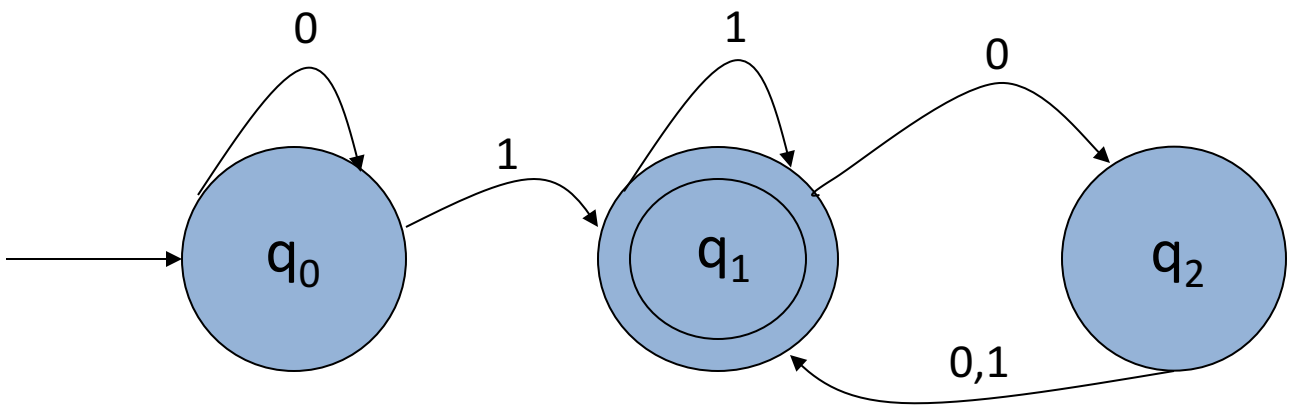
- State diagram
  - Start state q_0 (entering arrow from no state),
  - Accept state q_1 (double circle),
  - Transitions (arrows)
- M_1 will accept (or recognize) a string like "1101" if it ends in accept state. Otherwise the string is rejected!
- Can you describe all strings (i.e., a property shared by all and only the strings) that this automaton will accept?
  - It will accept all strings ending in a 1 and any string with an even number of 0's following the last 1

Formal Definition of Finite Automata

A finite automaton is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$

- Q is a finite set called states
- Σ is a finite set called the alphabet
- $\delta : Q \times \Sigma \rightarrow Q$ is the transition function
- $q_0 \in Q$ is the start state
- $F \subseteq Q$ is the set of accept states

Formal Definition



- $M_1 = (Q, \Sigma, \delta, q_0, F)$
  - $Q = \{q_0, q_1, q_2\}$
  - $\Sigma = \{0,1\}$
  - q_0 is the start state
  - $F = \{q_1\}$

Transition function δ

	0	**1**
q_0	q_0	q_1
q_1	q_2	q_1
q_2	q_1	q_1

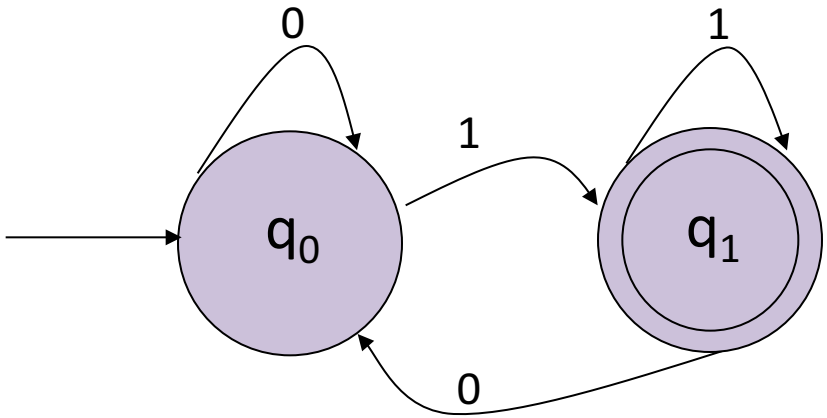
The Language of an automaton

- The language of DFA M is the set A of all strings accepted by the DFA M
  - L(M) = A
  - We also say that M recognizes A or M accepts A
- Convention: M accepts strings and recognizes a language
- Attention to quantifiers: a machine may accept many strings, but only one language
- Multiple automata can recognize the same language, but an automaton only recognizes a single language

What is the Language of M_1?

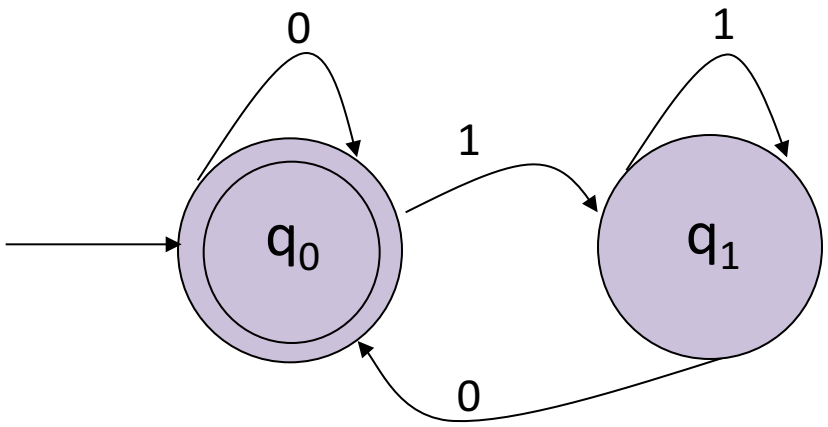
- $L(M_1)$ = A or M_1 recognizes A
- What is A?
  - A = {w | …….}
  - A = {w| w contains at least one 1 and an even number of 0's follows the last 1}

What is the Language of M_2?



- $M_2 = \{\{q_0,q_1\}, \{0,1\}, \delta, q_0, \{q_1\}\}$
  - I leave δ as an exercise
  - What is the language of M_2?
    - L(M2) = {w| ? }
    - L(M2) = {w| w ends in a 1}

What is the Language of M_3?



- M_3 is M_2 with a different accept state
- What is the language of M_3?
  - L(M3) = {w| ? }
  - L(M3) = {w| w ends in 0} [Not quite right! Why?]
  - L(M3) = {w| w is the empty string ε or ends in 0}

What is the Language of M_4?

What language does M_4 recognize?

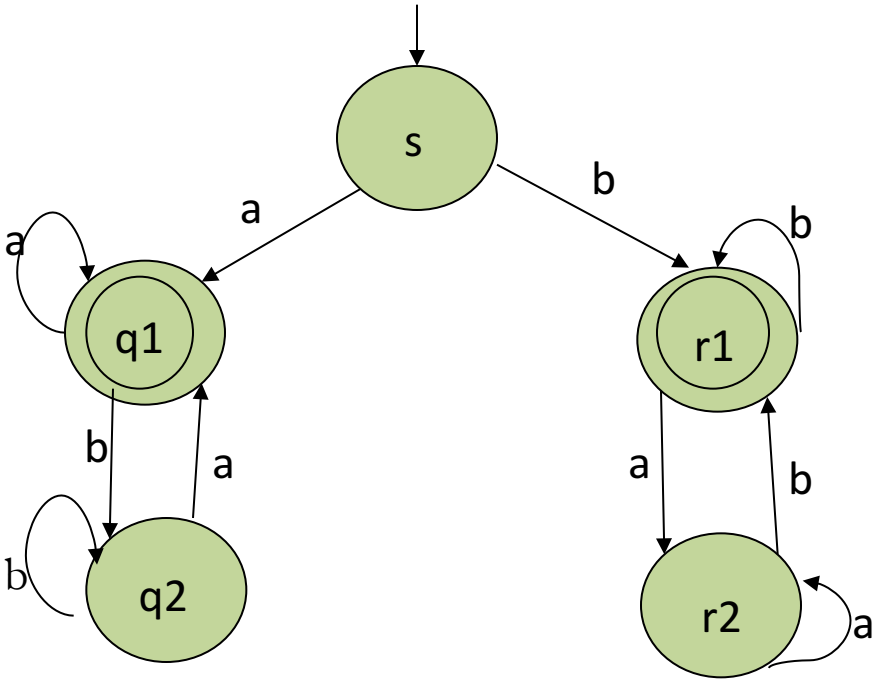


Figure 1.12 on page 38

What is the Language of M_4?

What language does M_4 recognize?

- M_4 accepts all strings that start and end with “a” or start and end with “b”
- More simply, language $L(M_4)$ is the set of all string starting and ending with the same symbol
  - Note that length 1 is allowed!

Construct M_5 to do Modulo-3 Arithmetic

- Alphabet $\sum$ = {RESET, 0, 1, 2}
- Construct M_5 to accept a string only if the sum of each input symbol modulo 3 is 0 and RESET sets the sum back to 0 (1.13, page 39)
  - ({q0, q1, q2} , {0,1,2,RESET}, δ_i, q_0, {q_0})
    - $\delta_i(q_j, 0) = q_j$
    - $\delta_i(q_j, 1) = q_k$ where k=j+1 modulo 3
    - $\delta_i(q_j, 2) = q_k$ where k=j+2 modulo 3
    - $\delta_i(q_j, \text{RESET}) = q_o$

Now Generalize M_5

- Generalize M_5 to accept if sum of symbols is a multiple of *i* instead of 3
  - $(\{q0, q1, q2, q3, \ldots, q_{i-1}\}, \{0,1,2,RESET\}, \delta_i, q_0, \{q_0\})$
    - $\delta_i(q_j, 0) = q_j$
    - $\delta_i(q_j, 1) = q_k$ where k=j+1 modulo i
    - $\delta_i(q_j, 2) = q_k$ where k=j+2 modulo i
    - $\delta_i(q_j, RESET) = q_o$
- Note: as long as *i* is finite, we only need finite memory (log # of states)
- Could you generalize on $\sum = \{1, 2, 3, \ldots k\}$?
  - Set of states would not change
  - More transitions!

Formal Definition of Accept

- Let $M = (Q, \Sigma, \delta, q_0, F)$ be a finite automaton and let $w=w_1w_2 \ldots w_n$ be a string where $w_i \in \Sigma$
- Then M accepts w if there exists a sequence of states $r_0, r_1, \ldots, r_n$ in Q such that:
  - $r_0=q_0$
  - $\delta(r_i, w_{i+1}) = r_{i+1}$, for i =0, 1, …, n-1
  - $r_n \in F$

Regular Languages

Definition: A language is called a regular language if there exists a finite automaton which recognizes it

- That is, if there exists a finite automaton that accepts all and only the strings in the language

Designing Finite Automata

- You will need to design FA which accept a given language
- Strategy:
  - Determine what you need to remember (the states)
    - E.g., How many states to determine even/odd number of 1's in an input?
    - What does each state represent
  - Set the start and accept states based on what each state represents
  - Assign the transitions
  - Check your solution: it should accept w $\in$ L and not accept w not in L
  - Be careful about the empty string

Designing FAs

ALWAYS start by asking: “What do I need to remember?”

- Design a FA to accept the language of binary strings where the number of 1’s is odd, zero counts as even (page 43)
- Design a FA to accept all string with 001 as a substring (page 44)
- Design a FA to accept a string with substring *abab*

Regular Operations

Let A and B be languages. We define 3 regular operations:

- Union: $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$
- Concatenation: $A \cdot B$ where $\{xy \mid x \in A \text{ and } y \in B\}$
- Star: $A^* = \{x_1 x_2 \ldots x_k \mid k \geq 0 \text{ and each } x_i \in A\}$
  - Star is a unary operator on a single language
  - Star repeats a string 0 or more times

Examples of Regular Operations

- Let A = {0, 1} and B = {c, d}
- Then:
  - $A \cup B = \{0, 1, c, d\}$
  - $A \cdot B = \{0c, 0d, 1c, 1d\}$
  - $A^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, \ldots\}$

Closure

- A collection of objects is closed under an operation if applying that operation to members of the collection returns an object in the collection

- E.g., The set of natural numbers is closed under addition and multiplication (but not division and subtraction)

Closure for Regular Languages

- Regular languages are closed under the regular operators union, concatenation and star
- Why we care?
  - If these operators are closed, then, if we can implement each operator using a FA, we can also build a FA to recognize any of their combinations

Closure of Union

Theorem 1.25: The class of regular languages is closed under the union operation

- If A_1 and A_2 are regular languages then so is $A_1 \cup A_2$
- How can we prove this? Use proof by construction:
  - Assume M_1 accepts A_1 and M_2 accepts A_2
  - Construct M_3 using M_1 and M_2 to accept $A_1 \cup A_2$
  - We need to simulate M_1 and M_2 running in parallel and stop if either reaches an accept state
    - This last part is feasible since we can have multiple accept states
    - You need to remember where you would be in both machines

Closure of Union

- You need to generate a state to represent the state you would be in for both M_1 and M_2
- Let $M_1=(Q_1, \Sigma, \delta_1, q_1, F_1)$ and $M_2=(Q_2, \Sigma, \delta_2, q_2, F_2)$
- Build $M_3=(Q, \Sigma, \delta, q_0, F)$ as follows :
  - $Q=\{(r_1,r_2) | r_1 \in Q_1$ and $r_2 \in Q_2\}$(Cartesian product $Q=Q_1 \times Q_2$)
    - Careful: order of (r1,r2) does not matter!
    - More correct: Q is the set of unordered pairs such that one state is in Q_1 and the other is in Q_2
  - $\Sigma = \Sigma_1 \cup \Sigma_2$
  - q_0 is the pair (q_1, q_2)
  - $F = \{(r_1, r_2) | r_1 \in F_1$ or $r_2 \in F_2\}$
  - $\delta((r_1,r_2),a) = (\delta(r_1, a), \delta_2(r_2, a))$

Closure of Concatenation

Theorem 1.26: The class of regular languages is closed under the concatenation operator

- If A_1 and A_2 are regular languages then so is $A_1 \cdot A_2$
- Can we construct a DFA which recognizes $A_1 \cdot A_2$?
- Can you see how to do this simply?
  - Since A_1 and A_2 are regular, there exists a DFA M_1 which recognizes A_1 and M_2 which recognizes A_2
  - Combining M_1 and M_2 is not trivial:
    - We cannot just concatenate M_1 and M_2, where start state of M_2 become the finish states of M_1
    - Because we do not accept a string as soon as it enters the finish state (wait until string is done) it can leave and come back
    - Thus we do not know when to start using M_2; if we make the wrong choice will not accept a string that can be accepted
    - This proof is easy if we have N ondeterministic FA (NFA)

Concatenation: Simple Example

Concatenation of the following:

- $L(M_1) = A$, where $\Sigma = \{0, 1\}$ and A = binary string with exactly 2 "1"' s
- $L(M_2) = B$, where $\Sigma = \{0, 1\}$ and B = binary string with exactly 3 "1"' s

- M_1 will enter accept state as soon as sees 2 "1"' s.
- It can then loop back on any "0"' s or move to M_2 without issue.
- It can move immediately to M_2 on a 1, and not have an issue since it cannot loop back
- Since A includes only strings with exactly 2 "1"' s. Once in M_2 everything will work okay.

Concatenation: Hard Example

- $L(M_1)$ = A, where Σ = {0, 1} and A = binary string with *at least* 2 "1"'s
- $L(M_2)$ = B, where Σ = {0, 1} and B = binary string with exactly 2 "1"'s
- Previous construction does not work (but easy with NFA or more complicated DFA)
  - If while in M_1 and see 2 "1"'s, enter accept state. When see another 1, have choice to loop back into accept state in M_1, or start moving into M_2, to the state that represents saw first 1 for string in B.
    - If the concatenated string has exactly 4 "1"'s total, then will only accept if move into M_2 as early as possible (after seeing the first 2 1's)
    - If the concatenated string has more than 4 "1"'s, then will only accept if loop in M_1 accept state until only 2 "1"'s left.